Dr. Dobb's Journal August 2008

The Many Faces of Deadlock

Despite the name, deadlock isn't (only) about locks

By Herb Sutter

Herb is a software development consultant, a software architect at Microsoft, and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

Quick: What is "deadlock"? How would you define it? One common answer is: "When two threads each try to take a lock the other already holds." Yes, we do indeed have a potential deadlock anytime two threads (or other concurrent tasks, such as work items running on a thread pool; for simplicity, I'll say "threads" for the purposes of this article) try to acquire the same two locks in opposite orders, and the threads might run concurrently. The potential deadlock will only manifest when the threads actually do run concurrently, and interleave in an appropriately bad way where each successfully takes its first lock and then waits forever on the other's, as in the execution *A->B->C->D* in the following example:

```
// Thread 1
mut1.lock();            // A
mut2.lock();            // C: blocks

// Thread 2
mut2.lock();            // B
mut1.lock();            // D: blocks
```

That's the classic deadlock example from college. Of course, two isn't a magic number. An improved definition of deadlock is: "When *N* threads enter a locking cycle where each tries to take a lock the next already holds."

Deadlock Among Messages

"But wait," someone might say. "I once had a deadlock just like the code you just showed, but it didn't involve locks at all—it involved messages." For example, consider this code, which is similar to the lock-based deadlock example:

```
// Thread 1
mq1.receive();          // blocks
mq2.send( x );

// Thread 2
mq2.receive();          // blocks
mq1.send( y );
```

Now Thread 1 is blocked, waiting for a message that Thread 2 hasn't sent yet. Unfortunately, Thread 2 is also blocked, waiting in turn for a message that Thread 1 hasn't sent yet. The result: A deadlock that is qualitatively the same as the one that arose from locks. So, then, what is a complete definition of deadlock?

Dead(b)locks

The key thing to recognize is that deadlock doesn't arise from a locking cycle exclusively. Any blocking (or, waiting) cycle will do. So a complete definition of deadlock could be put something like this:

> *deadlock, n.: When N concurrent tasks enter a cycle where each one is blocked waiting for the next.*

Clearly, blocking operations include all kinds of blocking synchronization: acquiring a lock (of course); waiting for a semaphore or condition variable to be signaled; waiting to join with another thread or task...But it also includes any other kind of call that waits for a result, including:

- Acquiring exclusive access to any resource (notably I/O, such as opening a file for writing).
- Waiting for a future or write-once variable to be available (the result of an asynchronous task).
- Waiting for any other kind of message to arrive from elsewhere, whether in-process (waiting for a queue container to become nonempty, waiting for a message to be placed into a GUI message queue) or out-of-process or out-of-box (performing a blocking receive call on a TCP socket, for instance).
- Anything else that waits for some other condition to be satisfied.

Complex Combinations

For a more complex example, consider Figure 1. There are four things going on:

- Thread 1 is a GUI window message handler that's in the middle of processing a message. As part of that processing, it needs to use some shared state and so wants to take a lock on mutex *mut*—which is currently held by Thread 4.
- Thread 4 is doing something with the same shared state and so has acquired *mut* already, and is just waiting to receive a message from a message queue—a message that is yet to be sent from Thread 2. (Doing communication inside a critical section is problematic and should be avoided where possible. See [2] for more about why to avoid doing communication while holding a lock.)
- Thread 2 hasn't got around to sending that message yet, because it's waiting to open a file for writing, which is an exclusive mode—a file currently opened for writing and appending on Thread 3.
- Thread 3 is working with the file, and posts a message to the window, which is a synchronous call that blocks to wait for the message to be handled—but Thread 1 is already handling a message and so can't pump and handle this one.

[Click image to view at full size]

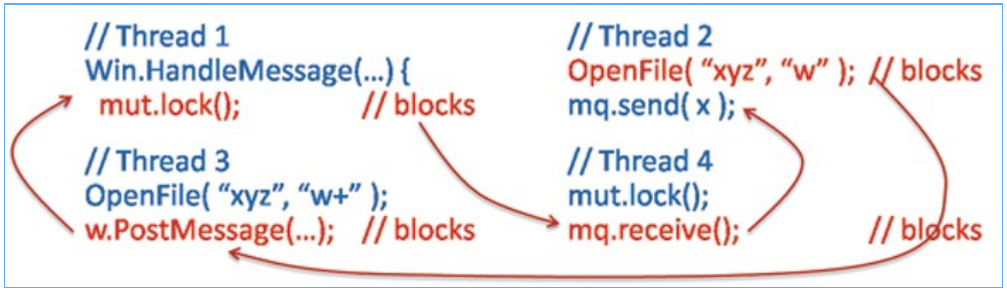

Figure 1: Sample deadlock among locks, message pumps, messages queues, and file access.

Finally, for extra fun and just to emphasize the point that any kind of blocking operation will do, consider that deadlock can involve a human—you! For example:

```
// Thread 1 (running in the CPU hardware)
mut.lock();
char = ReadKeystroke();                          // blocks
```

```
// Task 2 (running in your brain's wetware)
Wait for the prompt to appear.                  // blocks
OK, now start typing.

// Thread 3 (back in the hardware again)
mut1.lock();                       // blocks
Print ( "Press 'Any' key to continue..." );
```

Here, Thread 1 can't make progress because it's blocked waiting for you to press a key. You haven't pressed a key because you're still waiting for a prompt. Thread 3 can't print the prompt until it returns from being blocked waiting to lock a mutex...held by Thread 1. Around and around we go.

Dealing With Deadlock

The good news is that we have tools and techniques to detect and prevent many kinds of deadlock. In particular, consider ways to deal with two popular forms of blocking: Waiting to acquire a mutex, and waiting to receive a message.

First, to prevent locking cycles within the code you control, use lock hierarchies and other ordering techniques to ensure that your program always acquires all mutexes in the same order [1]. Note: Not all locks need to participate directly in a lock hierarchy; for example, some groups of related locks already have a total ordering among themselves, such as a chain of locks always traversed in the same order (hand-over-hand locking along a singly linked list, for instance).

Second, to prevent many kinds of message cycles, disciplines have been proposed to express contracts on communication channels, which helps to impose a known ordering on messages. One example is WS-CDL [3]. The general idea is to define rules for the orders in which messages must be sent and received, which can be enforced statically or dynamically to ensure that components communicating via messages won't tie themselves into a knot. A message ordering contract is typically expressed as some form of state machine. For example, here's how we might express a simple interface in pseudocode for a buyer requesting a purchase order:

```
contract PORequest {
// messages from requester to
// provider (arbitrarily
// choose "in" to be from the
// provider's point of view)
  in void Request( Info );

// messages from provider
// to requester
  out void Ack();
  out void Response( Details );

// now declare states and transitions
  state Start {
   Request -> RequestSent;
  }
  state RequestSent {
   Ack -> RequestReceived;
  }
  state RequestReceived {
   Response -> End;
```

```
  }
}
```

The only valid message order is *Request->Ack->Response.* If a provider could mistakenly send a *Response* without first sending an *Ack*, a requester could hang indefinitely waiting for the missing *Ack* message. Expressing the message order contract gives us a way to guarantee that a provider fulfills the contract, or at least to detect violations.

General Deadlock Detection

The bad news is that, as far as I know, there's no tool on the planet that identifies all kinds of blocking cycles for you, especially ones that consist of more than one kind of blocking. Lock hierarchies only guarantee freedom from deadlock among locks in the code you control; message contracts on communication channels only guarantee freedom from deadlock among messages.

Probably the best you can do today is to roll your own deadlock detection in code, by adopting a discipline like the following:

- Identify every condition or resource that can be waited for (a mutex, a message, a value of an atomic variable, an exclusive use of a file) and give it a unique name by creating a dummy object to represent it.
- Instrument every "start wait" and "end wait" point in your code by calling two helper functions: The first records that a condition or resource is about to be waited for (e.g,. StartWait(thing)) and should internally record the current thread's ID and the thing being waited for; it can also check to see if there is now a waiting cycle among threads and resources, and report the deadlock if that occurs. The second records that a wait has ended (e.g., EndWait(thing)) and will remove the waiting edge.

To illustrate how we can apply such a discipline, consider this deadlock between a mutex and a message that arises in the execution *A->B->C*:

```
// Global data
Mutex mut;
MessageQueue queue;

// Thread 1
mut.lock();             // A
queue.receive();                // B: blocks
mut.unlock();
// Thread 2
mut.lock();             // C: blocks
queue.send( msg );
```

We could apply a wait start/end instrumentation discipline as follows. Here the implementation of *StartWait* and *EndWait* is left for the reader, but should record which threads are waiting for which objects as described above:

```
// Global data
Mutex mut;
WaitableObject w_mut;
MessageQueue queue;
WaitableObject w_queue;
// Thread 1
StartWait( w_mut );
mut.lock();              // A
EndWait( w_mut );
```

```
StartWait( w_queue );
queue.receive();          // B: blocks
EndWait( w_queue );
mut.unlock();

// Thread 2
StartWait( w_mut );
mut.lock();               // C: blocks
EndWait( w_mut );
queue.send( msg );
```

That's a sketch of the idea. It's only a coding discipline, but it's an approach that can help you to instrument all waiting in a unified way, at least within the code you control.

Summary

Deadlock can arise whenever there is a blocking (or waiting) cycle among concurrent tasks, where each one is waiting for the next to produce some value or release some resource. Eliminate deadlocks as much as possible by applying ordering techniques like lock hierarchies and message contracts; these techniques are important, even though they are incomplete because each one deals with only a specific kind of waiting. Then consider adding your own deadlock detection by instrumenting the wait points in your code in a uniform way.

Whimsically, we might say that a more correct name for deadlock could be "deadblock"...but the world has already adopted a common spelling that's one letter shorter, and this isn't the time to try to change that. When reasoning about deadlock, remember not to forget the important "b", even though it's silent in pronunciation and in the common spelling.

Notes

[1] H. Sutter. "Use Lock Hierarchies To Avoid Deadlock" (*DDJ*, January 2008).

[2] H. Sutter. "Avoid Calling Unknown Code While Inside a Critical Section" (*DDJ*, December 2007).

[3] Web Services Choreography Description Language (WS-CDL) (*W3C*, 2005) (www.w3.org/TR/ws-cdl-10/).